

Week 16 Scala – PySpark equivalent programs

Week 16 is based on Spark structured streaming where we need real time stream. We will use socket and file as source for streaming

Generalized changes that are required in every program

1. Remove all val, var keyword as python does not have val and var types.
2. Anonymous functions are replaced with lambda in python.
3. Comment is given using # in python instead of // in scala
4. To write multi line code in pyspark, end every line with \ except the last line.

Note

1. There are be many ways to get the output for particular problem, we are showcasing one way.
2. Changes are highlighted in yellow.
3. Ncat is Linux utility. For windows, follow below steps.

Steps for streaming program execution on windows.

1. [Download the Free Nmap Security Scanner for Linux/Mac/Windows](#) Download nmap-stable setup and install
2. In code give localhost 9998
3. Run the code... It will give error because no port is listening ... That's ok
4. Open cmd – go to Nmap folder
5. ncat -lvp 9998
6. start typing words
7. Cross check in your program

```
val ssc = new StreamingContext(sc, Seconds(5))
//lines is a dstream
val lines = ssc.socketTextStream("localhost",9998)
//words is a transformed dstream
val words = lines.flatMap(x => x.split(" "))
val pairs = words.map(x => (x,1))
val wordCounts = pairs.reduceByKey((x,y) => x+y)
wordCounts.print()
ssc.start()
```

```
ncat: Listening on 0.0.0.0:9998
ncat: Connection from 192.168.59.1.
ncat: Connection from 192.168.59.1:9376.
hi
kk^C
C:\Program Files (x86)\Nmap\ncat -lvp 9998
ncat: Version 7.92 ( https://nmap.org/ncat )
ncat: Listening on :::9998
ncat: Listening on 0.0.0.0:9998
ncat: Connection from 127.0.0.1.
ncat: Connection from 127.0.0.1:24771.
hi
hi
hi
hi hi hi
```

```
Command Prompt - spark-shell --master local[*]
[Stage 0:> (0 + 1) / 1]21/12/01 18:10:01 WARN RandomBlockReplicationPolicy: Expecting 1 r
eplicas with only 0 peer/s.
21/12/01 18:10:01 WARN BlockManager: Block input-0-1638362401600 replicat
ed to only 0 peer(s) instead of 1 peers
-----
Time: 1638362405000 ms.
-----
(hi,1)
[Stage 0:> (0 + 1) / 1]21/12/01 18:10:07 WARN RandomBlockReplicationPolicy: Expecting 1 r
eplicas with only 0 peer/s.
21/12/01 18:10:07 WARN BlockManager: Block input-0-1638362407200 replicat
ed to only 0 peer(s) instead of 1 peers
-----
Time: 1638362410000 ms.
-----
(hi,1)
21/12/01 18:10:10 WARN RandomBlockReplicationPolicy: Expecting 1 replicas
with only 0 peer/s.
21/12/01 18:10:10 WARN BlockManager: Block input-0-1638362410200 replicat
ed to only 0 peer(s) instead of 1 peers
[Stage 0:> (0 + 1) / 1]
-----
Time: 1638362415000 ms.
-----
(hi,3)
[Stage 0:> (0 + 1) / 1]
```

TECH
REER!

TRENDY TECH

Problem Statement: Write a real time word count program using spark structured stream

Solution:

Scala Spark Program	PySpark Program
<pre>import org.apache.spark.sql.SparkSession val spark = SparkSession.builder .master("local[2]") .appName("My Streaming Application") .getOrCreate() // read from the stream val linesDf = spark.readStream .format("socket") .option("host", "localhost") .option("port", "12345") .load() linesDf.printSchema() // process val wordsDf=linesDf.selectExpr("explode(split(value,' ')) as word") val countsDf =wordsDf.groupBy("word").count() // write to the sink val wordCountQuery = countsDf.writeStream .format('console') .outputMode('complete') .option("checkpointLocation","checkpoint-location1") .start() wordCountQuery.awaitTermination()</pre>	<pre>from pyspark.sql import SparkSession spark = SparkSession.builder \ .master("local[2]") \ .appName("My Streaming Application") \ .getOrCreate() # read from the stream linesDf = spark.readStream \ .format("socket") \ .option("host", "localhost") \ .option("port", "12345") \ .load() \ linesDf.printSchema() # process wordsDf=linesDf.selectExpr("explode(split(value,' ')) as word") countsDf =wordsDf.groupBy("word").count() # write to the sink wordCountQuery = countsDf.writeStream \ .format('console') \ .outputMode('complete') \ .option("checkpointLocation","checkpoint-location1") \ .start() wordCountQuery.awaitTermination()</pre>
Add below two bold lines and run again, you can see the performance gain. NO change in pyspark	

TRENDY TECH

<pre>spark = SparkSession.builder .master("local[2]") .appName("My Streaming Application") .config("spark.sql.shuffle.partitions",3) .config("spark.streaming.stopGracefullyOnShutdown","true") .getOrCreate()</pre>	<pre>spark = SparkSession.builder \ .master("local[2]") \ .appName("My Streaming Application") \ .config("spark.sql.shuffle.partitions",3) \ .config("spark.streaming.stopGracefullyOnShutdown","true") \ .getOrCreate()</pre>
Add below bold line to Set triggering time. Small difference in pyspark	
<pre>wordCountQuery = countsDf.writeStream .format('console') .outputMode('complete') .option("checkpointLocation","checkpoint-location2") .trigger(Trigger.ProcessingTime("5 seconds")) .start()</pre>	<pre>wordCountQuery = countsDf.writeStream \ .format('console') \ .outputMode('complete') \ .option("checkpointLocation","checkpoint-location2") \ .trigger(processingTime='5 seconds') \ .start()</pre>

Specific changes that are required in above program

1. Import is different
2. Need to remove all val keywords
3. Add \ at end of line to continue code to next line.
4. `.trigger(Trigger.ProcessingTime("5 seconds"))` in scala is changed to `.trigger(processingTime='5 seconds') \` in pyspark

UPLIFT YOUR CAREER!

Problem Statement: Read json file and write json file using spark structured streaming

Solution:

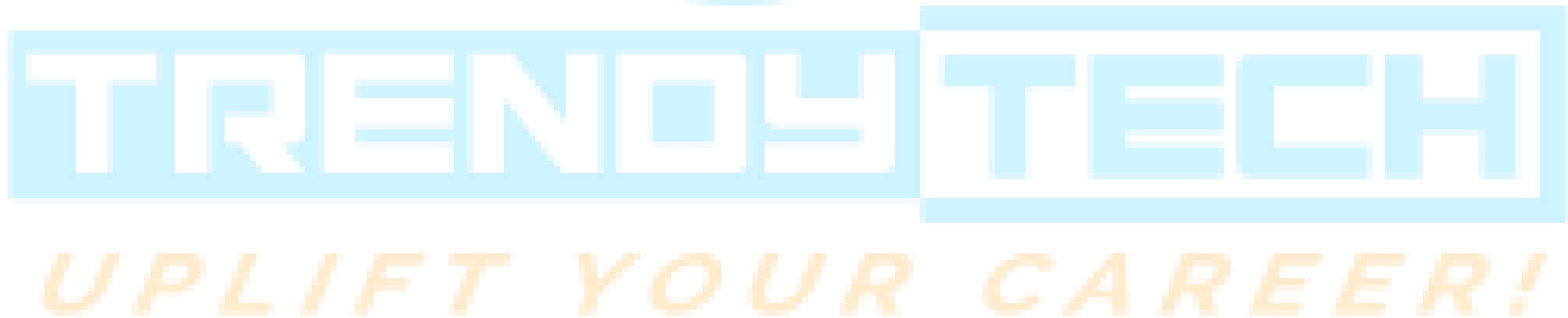
Scala Spark Program	PySpark Program
<pre>import org.apache.spark.sql.SparkSession val spark = SparkSession.builder .master("local[2]") .appName("My Streaming Application") .config("spark.sql.shuffle.partitions",3) .config("spark.streaming.stopGracefullyOnShutdown","true") .config("spark.sql.streaming.schemaInference","true") .getOrCreate() // read from file source val ordersDf = spark.readStream .format("json") .option("path", "myinputfolder") .load() // process val ordersDf.createOrReplaceTempView("orders") val completeOrders =spark.sql("select * from orders where order_status='COMPLETE'") // write to the sink val wordCountQuery = completeOrders.writeStream .format('json') .outputMode('append') .option("path", "myoutputfolder") .option("checkpointLocation","checkpoint-location5") .trigger(Trigger.ProcessingTime("5 seconds"))</pre>	<pre>from pyspark.sql import SparkSession spark = SparkSession.builder \ .master("local[2]") \ .appName("My Streaming Application") \ .config("spark.sql.shuffle.partitions",3) \ .config("spark.streaming.stopGracefullyOnShutdown","true") \ .config("spark.sql.streaming.schemaInference","true") \ .getOrCreate() # read from file source ordersDf = spark.readStream \ .format("json") \ .option("path", "myinputfolder") \ .load() # process ordersDf.createOrReplaceTempView("orders") completeOrders =spark.sql("select * from orders where order_status='COMPLETE'") # write to the sink wordCountQuery = completeOrders.writeStream \ .format('json') \ .outputMode('append') \ .option("path", "myoutputfolder") \ .option("checkpointLocation","checkpoint-location5") \ .trigger(processingTime='30 seconds') \</pre>

TRENDY TECH

<code>.start()</code> <code>wordCountQuery.awaitTermination()</code>	<code>.start()</code> <code>wordCountQuery.awaitTermination()</code>
Add bold line to set micro batch of one file.	NO change in pyspark
<code>val ordersDf = spark.readStream</code> <code>.format("json")</code> <code>.option("path", "myinputfolder")</code> <code>.option("maxFilesPerTrigger",1)</code> <code>.load()</code>	<code>ordersDf = spark.readStream \</code> <code>.format("json") \</code> <code>.option("path", "myinputfolder") \</code> <code>.option("maxFilesPerTrigger",1) \</code> <code>.load()\</code>

Specific changes that are required in above program

1. Import is different
2. Need to remove all val keywords
3. Add \ at end of line to continue code to next line.
4. `.trigger(Trigger.ProcessingTime("5 seconds"))` in scala is changed to `.trigger(processingTime='5 seconds') \` in pyspark



TRENDY TECH

Problem Statement: Write a real time tumbling window program to read json file.

Solution:

Scala Spark Program	PySpark Program
<pre>import org.apache.spark.SparkConf import org.apache.spark.sql.SparkSession import org.apache.log4j.Level import org.apache.log4j.Logger import java.sql.Timestamp import org.apache.spark.sql.types.StructType import org.apache.spark.sql.types.StructField import org.apache.spark.sql.types.IntegerType import org.apache.spark.sql.types.StringType import org.apache.spark.sql.functions._ import org.apache.spark.sql.streaming.Trigger object TumblingDemo extends App { Logger.getLogger("org").setLevel(Level.ERROR) val spark = SparkSession.builder() .master("local[2]") .appName("Tumbling") .config("spark.streaming.stopGracefullyOnShutdown","true") .config("spark.sql.shuffle.partitions",3) .getOrCreate() //1. read val ordersDf = spark.readStream .format("socket") .option("host","localhost") .option("port","12345") .load //define own schema instead of inferring it val orderSchema= StructType(List(</pre>	<pre>from pyspark.sql import SparkSession from pyspark.sql.functions import from_json, window from pyspark.sql.types import StructType, StructField, IntegerType , StringType from pyspark.sql import functions as F spark = SparkSession.builder \ .master("local[2]") \ .appName("Tumbling") \ .config("spark.streaming.stopGracefullyOnShutdown", "true") \ .config("spark.sql.shuffle.partitions", 3) \ .getOrCreate() # 1. read ordersDf = spark.readStream \ .format("socket") \ .option("host", "localhost") \ .option("port", "12345") \ .load() # define own schema instead of inferring it orderSchema = StructType([</pre>

TRENDY TECH

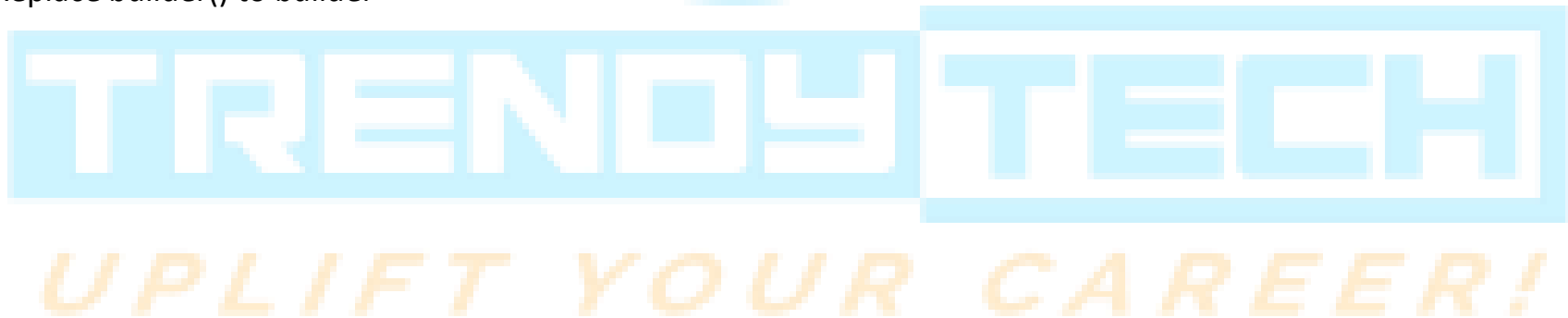
<pre> StructField("order_id", IntegerType), StructField("order_date", StringType), StructField("order_customer_id", IntegerType), StructField("order_status", StringType), StructField("amount", IntegerType),)) //2. process val valueDF = ordersDf.select(from_json(col("value"), orderSchema).alias("value")) val refinedOrderDF = valueDF.select("value.*") val windowAggDF = refinedOrderDF .groupBy(window(col("order_date"), "15 minute")) .agg(sum("amount").alias("totalInvoice")) val opDf= windowAggDF.select("window.start", "window.end", "totalInvoice") val ordersQuery = opDf.writeStream .format("console") .outputMode("update") .option("checkpointLocation", "chk-Loc1") .trigger(Trigger.ProcessingTime("15 second")) .start() ordersQuery.awaitTermination() } </pre>	<pre> StructField("order_id", IntegerType()), StructField("order_date", StringType()), StructField("order_customer_id", IntegerType()), StructField("order_status", StringType()), StructField("amount", IntegerType()),) # 2. process valueDF = ordersDf.select(from_json(F.col("value"), orderSchema).alias("value")) refinedOrderDF = valueDF.select("value.*") windowAggDF = refinedOrderDF \ .groupBy(window(F.col("order_date"), "15 minute")) \ .agg(F.sum("amount").alias("totalInvoice")) opDf = windowAggDF.select("window.start", "window.end", "totalInvoice") ordersQuery = opDf.writeStream \ .format("console") \ .outputMode("update") \ .option("checkpointLocation", "chk-Loc1") \ .trigger(processingTime="15 second")\ .start() ordersQuery.awaitTermination() </pre>
Add bold line to use Watermark	NO change in pyspark
<pre> val windowAggDF = refinedOrderDF .withWatermark("order_date", "30 minute") .groupBy(window(col("order_date"), "15 minute")) .agg(sum("amount") .alias("totalInvoice")) </pre>	<pre> windowAggDF = refinedOrderDF \ .withWatermark("order_date", "30 minute") \ .groupBy(window(F.col("order_date"), "15 minute")) \ .agg(F.sum("amount").alias("totalInvoice")) </pre>

TRENDY TECH

Add bold argument to make the window sliding	NO change in pyspark
<pre>windowAggDF = refinedOrderDF .withWatermark("order_date","30 minute").groupBy(window(F.col("order_date"), "15 minute", "1 minute")) .agg(F.sum("amount").alias("totalInvoice"))</pre>	<pre>windowAggDF = refinedOrderDF \ .withWatermark("order_date","30 minute") \ .groupBy(window(F.col("order_date"), "15 minute", "1 minute")) \ .agg(F.sum("amount").alias("totalInvoice"))</pre>

Specific changes that are required in above program

1. Import is different
2. Need to remove all val keywords
3. Add \ at end of line to continue code to next line.
4. .trigger(Trigger.ProcessingTime("15 seconds")) in scala is changed to .trigger(processingTime='15 seconds') \ in pyspark
5. In pyspark some functions such as col and sum cannot be used directly hence we add **from pyspark.sql import functions as F** and then use **F.col** or **F.sum**
6. List() is changed to []
7. Replace load with load(), same for IntegerType(),StringType()
8. Replace builder() to builder



Problem Statement: Write a real time join program using spark structured streaming

Solution:

Scala Spark Program	PySpark Program
<pre>import org.apache.spark.sql.SparkSession import org.apache.log4j.Level import org.apache.log4j.Logger import java.sql.Timestamp import org.apache.spark.sql.types.StructType import org.apache.spark.sql.types.StructField import org.apache.spark.sql.types.TimestampType import org.apache.spark.sql.types.StringType import org.apache.spark.sql.functions._ import org.apache.spark.sql.streaming.Trigger val spark = SparkSession.builder() .master("local[2]") .appName("join") .config("spark.streaming.stopGracefullyOnShutdown", "true") .config("spark.sql.shuffle.partitions", 3) .getOrCreate() // define own schema instead of inferring it val impressionSchema = StructType(List(StructField("impressionID", StringType), StructField("ImpressionTime", TimestampType), StructField("CampaignName", StringType),)) val clickSchema = StructType(List(StructField("clickID", StringType), StructField("ClickTime", TimestampType),))</pre>	<pre>from pyspark.sql import SparkSession from pyspark.sql.functions import from_json, window from pyspark.sql.types import StructType, StructField, IntegerType, StringType, TimestampType from pyspark.sql import functions as F spark = SparkSession.builder \ .master("local[2]") \ .appName("join") \ .config("spark.streaming.stopGracefullyOnShutdown", "true") \ .config("spark.sql.shuffle.partitions", 3) \ .getOrCreate() # define own schema instead of inferring it impressionSchema = StructType([StructField("impressionID", StringType()), StructField("ImpressionTime", TimestampType()), StructField("CampaignName", StringType()),]) clickSchema = StructType([StructField("clickID", StringType()), StructField("ClickTime", TimestampType()),])</pre>

TRENDY TECH

```
// read the stream
val impressionsDf = spark.readStream
  .format("socket")
  .option("host", "localhost")
  .option("port", "12342")
  .load()

val clicksDf = spark.readStream
  .format("socket")
  .option("host", "localhost")
  .option("port", "12343")
  .load()

// structure the data based on the schema defined - impressionDf
val valueDF1 = impressionsDf.select(from_json(col("value"),
impressionSchema).alias("value"))
val impressionDfNew = valueDF1.select("value.*")

// structure the data based on the schema defined - clickDf
val valueDF2 = clicksDf.select(from_json(col("value"),
clickSchema).alias("value"))
val clickDfNew = valueDF2.select("value.*")

// join condition
val joinExpr = impressionDfNew.col("ImpressionID") ===
clickDfNew.col("clickID")

// join type
val joinType="inner"

// joining both the streaming data frames
val joinedDf = impressionDfNew.join(clickDfNew,joinExpr,joinType)
  .drop(clickDfNew.col("clickID"))

// output to the sink
val campaignQuery = joinedDf.writeStream
  .format("console")
```

```
# read the stream
impressionsDf = spark.readStream \
  .format("socket") \
  .option("host", "localhost") \
  .option("port", "12342") \
  .load()

clicksDf = spark.readStream \
  .format("socket") \
  .option("host", "localhost") \
  .option("port", "12343") \
  .load()

# structure the data based on the schema defined - impressionDf
valueDF1 = impressionsDf.select(from_json(F.col("value"),
impressionSchema).alias("value"))
impressionDfNew = valueDF1.select("value.*")

# structure the data based on the schema defined - clickDf
valueDF2 = clicksDf.select(from_json(F.col("value"),
clickSchema).alias("value"))
clickDfNew = valueDF2.select("value.*")

#join condition
joinExpr = impressionDfNew["ImpressionID"] == clickDfNew["clickID"]

#join type
joinType="inner"

#joining both the streaming data frames
joinedDf = impressionDfNew.join(clickDfNew,joinExpr,joinType) \
  .drop(clickDfNew["clickID"])

#output to the sink
campaignQuery = joinedDf.writeStream \
  .format("console") \
```

TRENDY TECH

<pre>.outputMode("append") .option("checkpointLocation", "chk-Loc2") .trigger(processingTime="15 second") .start() campaignQuery.awaitTermination()</pre>	<pre>.outputMode("append") \ .option("checkpointLocation", "chk-Loc2") \ .trigger(processingTime="15 second")\ .start() campaignQuery.awaitTermination()</pre>
Add bold code to use watermark	No change in pyspark except F.col
<pre>// structure the data based on the schema defined - impressionDf valueDF1 = impressionsDf.select(from_json(col("value"), impressionSchema).alias("value")) impressionDfNew = valueDF1.select("value.*").withWatermark("impressionTime", "30 minute") // structure the data based on the schema defined - clickDf valueDF2 = clicksDf.select(from_json(col("value"), clickSchema).alias("value")) clickDfNew = valueDF2.select("value.*").withWatermark("clickTime", "30 minute")</pre>	<pre># structure the data based on the schema defined - impressionDf valueDF1 = impressionsDf.select(from_json(F.col("value"), impressionSchema).alias("value")) impressionDfNew = valueDF1.select("value.*").withWatermark("impressionTime", "30 minute") # structure the data based on the schema defined - clickDf valueDF2 = clicksDf.select(from_json(F.col("value"), clickSchema).alias("value")) clickDfNew = valueDF2.select("value.*").withWatermark("clickTime", "30 minute")</pre>

Specific changes that are required in above program

1. Import is different
2. `.trigger(Trigger.ProcessingTime("15 seconds"))` in scala is changed to `.trigger(processingTime='15 seconds') \` in pyspark
3. In pyspark some functions such as `col` cannot be used directly hence we add `from pyspark.sql import functions as F` and then use `F.col`
4. `List()` is changed to `[]`
5. Replace `load` with `load()`, same for `IntegerType()`, `StringType()`
6. Replace `builder()` to `builder`
7. In pyspark dataframe does not have `col` function, hence we give column name using `[]`. Also Replace `===` with `==`

```
impressionDfNew["ImpressionID"] == clickDfNew["clickID"]
```